

2. 一维、二维数据与 CSV 格式

数据的组织形式

一维数据

- 一维数据是由一组地位相同的数据构成的序列
- 在 Python 中，一维数据通常用列表表示

二维数据

- 二维数据由多个一维数据构成，呈表格形式：每一行是一条记录，每一列是一个属性
- 在 Python 中，二维数据通常用嵌套列表表示：外层列表的每个元素是一行，每一行本身又是一个列表

嵌套结构数据

- 当一条记录中的某个属性本身又包含多个值时，数据就形成了多层嵌套的结构
- 在 Python 中，嵌套结构数据通常用字典与列表相互嵌套来表示

数据的组织形式

示例代码

```
# 一维数据：张三选修的课程
courses = ["Python程序设计", "大学英语", "高等数学"]

# 二维数据：选课名单，第一行为表头
students = [
    ["姓名", "学号", "专业"],
    ["张三", "2024001", "计算机科学与技术"],
    ["李四", "2024002", "软件工程"],
    ["王五", "2024003", "数据科学与大数据技术"]
]

# 嵌套结构数据：学生信息中包含选修课程列表
student = {
    "姓名": "张三",
    "学号": "2024001",
    "选修课程": ["Python程序设计", "线性代数"]
}
```

数据的组织形式

三类数据的对比

对比项	一维数据	二维数据	嵌套结构数据
结构	一组地位相同的数据	多行多列的表格	字典与列表多层嵌套
Python 中的表示	列表	嵌套列表	字典与列表相互嵌套
示例	一名学生的选修课程	选课名单	包含选修课程的学生信息
常用的文件存储格式	用分隔符分隔的文本	CSV 格式	JSON 格式

一维数据的存储

- **存储方法**
  - 一维数据存入文件时，需要选定一个分隔符，用它将各个数据隔开
- **示例一：以逗号为分隔符，将选修课程写入 CSV 文件**
  - 以逗号作为分隔符存储数据的格式，称为 CSV 格式
  - 这类文件的扩展名通常为 .csv
  - 一维数据以逗号分隔后写入文件，就构成了 CSV 文件中的一行

一维数据的存储

示例一：以逗号为分隔符，将选修课程写入 CSV 文件

```
courses = ["Python程序设计", "大学英语", "高等数学"]

# 写入：用逗号将各门课程连接成一个字符串，写入 CSV 文件
with open("courses.csv", "w", encoding="utf-8") as file:
    file.write(",".join(courses))

# 读取：读出整个字符串，再按逗号拆分为列表
with open("courses.csv", "r", encoding="utf-8") as file:
    content = file.read()
result = content.split(",")

print(content)          # 文件中的实际内容
print(result)           # 拆分后得到的列表
print(len(result))      # 列表中的元素个数
```

运行结果：

```
Python程序设计,大学英语,高等数学
['Python程序设计', '大学英语', '高等数学']
3
```

CSV格式

CSV 格式

- 以选课名单为例，文件 students.csv 的内容如下：

```
姓名,学号,专业
张三,2024001,计算机科学与技术
李四,2024002,软件工程
王五,2024003,数据科学与大数据技术
```

- 用 Excel 软件打开效果：

姓名	学号	专业	
张三	2024001	计算机科学与技术	
李四	2024002	软件工程	
王五	2024003	数据科学与大数据技术	

CSV格式

CSV 格式的基本规则

1. 纯文本格式，整个文件采用同一种字符编码
2. 以行为单位，每行表示一条记录；文件开头不留空行，行与行之间也没有空行
3. 每行中的各列数据之间用逗号分隔，逗号前后不加空格
4. 每行的列数相同。某列数据为空时，也要保留该列的逗号
5. 可以包含表头，也可以不包含。包含表头时，表头放在文件的第一行

CSV格式

示例一：逗号后加空格 (针对规则3)

- 文件 students_space.csv 中，每个逗号后都多写了一个空格：

```
姓名， 学号， 专业
张三， 2024001， 计算机科学与技术
李四， 2024002， 软件工程
王五， 2024003， 数据科学与大数据技术
```

```
with open("students_space.csv", "r", encoding="utf-8") as file:
    header = file.readline().strip().split(",")     # 读取表头，去掉换行符后按逗号拆分
    first = file.readline().strip().split(",")      # 读取第一条记录，处理方式相同
print(header)                          # 拆分后得到的列名
print(first)                           # 拆分后得到的第一条记录
print(header[1] == "学号")             # 判断第 2 个列名是否为“学号”
```

运行结果：

['姓名', ' 学号', ' 专业’]
['张三', ' 2024001', ' 计算机科学与技术’]
False

⚠ " 学号" 与 "学号" 是两个不同的字符串，比较结果为 False

CSV格式

- **示例二：空数据省略了逗号(针对规则4)**
  - 文件 students_empty.csv 中，李四和王五都尚未确定专业：李四一行保留了末尾的逗号（正确），王五一行省略了末尾的逗号（错误）

```
姓名,学号,专业
张三,2024001,计算机科学与技术
李四,2024002,
王五,2024003
```

CSV格式

示例二：空数据省略了逗号(针对规则4)

```
with open("students_empty.csv", "r", encoding="utf-8") as file:
    file.readline()                                    # 读取表头，不作处理，相当于跳过
    for line in file:
        fields = line.strip().split(",")               # 去掉换行符后按逗号拆分
        print("{}: 共{}列，专业为“{}”".format(fields[0], len(fields), fields[2]))
```

运行结果：

张三：共3列，专业为“计算机科学与技术”
李四：共3列，专业为“”
IndexError: list index out of range

王五一行省略了逗号，拆分后只有 2 列，访问第 3 列 fields[2] 时下标越界，引发 IndexError

CSV格式

- **读取 CSV 文件**
  - CSV 文件中的每一行对应一条记录
  - 逐行读取文件后，按逗号拆分每一行，得到这条记录的各列数据
  - 把各行的拆分结果依次放入一个列表，便得到一个二维列表

```
二维列表 = []
with open(文件路径, "r", encoding=编码名称) as 文件对象:
    for 行 in 文件对象:
        二维列表.append(行.strip().split(","))
```

CSV格式

示例：读取 CSV 文件

- 沿用选课名单文件 students.csv

```
姓名,学号,专业
张三,2024001,计算机科学与技术
李四,2024002,软件工程
王五,2024003,数据科学与大数据技术
```

```
students = []
with open("students.csv", "r", encoding="utf-8") as file:
    for line in file:
        students.append(line.strip().split(","))     # 去掉换行符后按逗号拆分，存入列表
```

CSV格式

- **写入 CSV 文件**
  - 写入二维列表时，对其中的每一行，先用逗号将各列数据连接成一个字符串
  - 再在末尾加上换行符，写入文件

```
with open(文件路径, "w", encoding=编码名称) as 文件对象:
    for 行 in 二维列表:
        文件对象.write(",".join(行) + "\n")
```

CSV格式

- **示例：数据中含有整数时直接连接**
  - 统计各门课程的选课人数后，将结果写入 course_count.csv
  - 选课人数是统计得到的整数

```
counts = [["课程", "选课人数"], ["Python程序设计", 3], ["大学英语", 2], ["高等数学", 1]]
with open("course_count.csv", "w", encoding="utf-8") as file:
    for row in counts:
        file.write(",".join(row) + "\n")     # 用逗号连接各列，末尾加换行符
```

运行结果：

TypeError: sequence item 1: expected str instance, int found

join() 要求列表中的每个元素都是字符串。["Python程序设计", 3] 的第 2 个元素是整数，引发 TypeError

出错前，表头一行已经写入了文件，因此 course_count.csv 中只有 课程,选课人数 一行

CSV格式

- **示例：先将各列转换为字符串，再连接**

```
counts = [["课程", "选课人数"], ["Python程序设计", 3],
          ["大学英语", 2], ["高等数学", 1]]

with open("course_count.csv", "w", encoding="utf-8") as file:
    for row in counts:
        # 将每一列转换为字符串，用逗号连接，末尾加换行符
        file.write(",".join(map(str, row)) + "\n")

with open("course_count.csv", "r", encoding="utf-8") as file:
    print(file.read())  # 查看文件的实际内容
```

运行结果：

课程,选课人数
Python程序设计,3
大学英语,2
高等数学,1

字符串方法的局限

- **问题：数据内部含有逗号**
  - 用 join() 和 split() 读写 CSV 文件时，逗号是唯一的分隔依据
  - 只要某个数据的内部也含有逗号，该数据在读取时就会被拆开
  - 同一行的列数随之改变，各列的对应关系也被打乱

字符串方法的局限

示例：选修课程中含有逗号

```
selections = [
    ["姓名", "学号", "选修课程"],
    ["张三", "2024001", "Python程序设计,大学英语"],
    ["李四", "2024002", "高等数学"],
    ["王五", "2024003", "线性代数,数据结构,大学物理"]
]

# 写入：用逗号连接各列，每行末尾加换行符
with open("selections.csv", "w", encoding="utf-8") as file:
    for row in selections:
        file.write(",".join(row) + "\n")

# 查看文件的实际内容
with open("selections.csv", "r", encoding="utf-8") as file:
    print(file.read())

# 读取：跳过表头，逐行按逗号拆分，输出列数和选修课程
with open("selections.csv", "r", encoding="utf-8") as file:
    file.readline()
    for line in file:
        fields = line.strip().split(",")
        print("{}：共{}列，选修课程为{}".format(fields[0], len(fields), fields[2]))
```

运行结果：

```
姓名,学号,选修课程
张三,2024001,Python程序设计,大学英语
李四,2024002,高等数学
王五,2024003,线性代数,数据结构,大学物理

张三：共4列，选修课程为Python程序设计
李四：共3列，选修课程为高等数学
王五：共5列，选修课程为线性代数
```

要正确读写这类数据，需要一种能够识别 CSV 格式规则的工具。Python 标准库中的 csv 模块提供了这一功能

csv 模块

写入 CSV 文件

- 先用 csv.writer() 根据文件对象创建一个写入器，再调用写入器的方法写入数据：

方法	语法格式	功能
writerow()	写入器.writerow(列表)	写入一行，列表中的每个元素为一列
writerows()	写入器.writerows(二维列表)	写入多行，二维列表中的每个元素为一行

语法格式

newline= “” 的作用是关闭 Python 对换行符的自动转换。使用 csv 模块读写文件时，建议总是加上 newline=""

```
import csv
with open(文件路径, "w", encoding=编码名称, newline="") as 文件对象:
    写入器 = csv.writer(文件对象)
    写入器.writerow(一行数据)
    写入器.writerows(多行数据)
```

csv 模块

示例：用 csv 模块写入选课记录

```
import csv

header = ["姓名", "学号", "选修课程", "课程数"]
selections = [
    ["张三", "2024001", "Python程序设计,大学英语", 2],
    ["李四", "2024002", "高等数学", 1],
    ["王五", "2024003", "线性代数,数据结构,大学物理", 3]
]

# 写入：先写表头，再写全部记录
with open("selections.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)          # 创建写入器
    writer.writerow(header)            # 写入一行：表头
    writer.writerows(selections)       # 写入多行：全部选课记录

# 查看文件的实际内容
with open("selections.csv", "r", encoding="utf-8") as file:
    print(file.read())
```

运行结果：

```
姓名,学号,选修课程,课程数
张三,2024001,"Python程序设计,大学英语",2
李四,2024002,高等数学,1
王五,2024003,"线性代数,数据结构,大学物理",3
```

⚠ 张三和王五的选修课程中含有逗号，写入器自动用双引号将其括起

csv 模块

示例：用 csv 模块写入选课记录

```
import csv

header = ["姓名", "学号", "选修课程", "课程数"]
selections = [
    ["张三", "2024001", "Python程序设计,大学英语", 2],
    ["李四", "2024002", "高等数学", 1],
    ["王五", "2024003", "线性代数,数据结构,大学物理", 3]
]

# 写入：先写表头，再写全部记录
with open("selections.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)          # 创建写入器
    writer.writerow(header)            # 写入一行：表头
    writer.writerows(selections)       # 写入多行：全部选课记录

# 查看文件的实际内容
with open("selections.csv", "r", encoding="utf-8") as file:
    print(file.read())
```

运行结果：

```
姓名,学号,选修课程,课程数
张三,2024001,"Python程序设计,大学英语",2
李四,2024002,高等数学,1
王五,2024003,"线性代数,数据结构,大学物理",3
```

⚠ “课程数”一列是整数，写入器自动将其转换为字符串后写入，不会像 join() 那样引发 TypeError

csv 模块

- **读取 CSV 文件**
  - 用 csv.reader() 根据文件对象创建一个读取器，再用 for 循环遍历读取器，每次得到一行数据组成的列表
- **语法格式**

```
import csv
with open(文件路径, "r", encoding=编码名称, newline="") as 文件对象:
    读取器 = csv.reader(文件对象)
    for 行 in 读取器:
        处理一行数据的语句块
```

csv 模块

示例：用 csv 模块读取选课记录

```
import csv

with open("selections.csv", "r", encoding="utf-8", newline="") as file:
    reader = csv.reader(file)          # 创建读取器
    for row in reader:
        print(row)                     # 每行数据组成一个列表
```

运行结果：

['姓名', '学号', '选修课程', '课程数']
['张三', '2024001', 'Python程序设计,大学英语', '2']
['李四', '2024002', '高等数学', '1']
['王五', '2024003', '线性代数,数据结构,大学物理', '3']

每行都正确地得到 4 列
双引号内的逗号被识别为数据的一部分，双引号本身不出现在结果中
读取器已去除行末的换行符，无须再调用 strip()
读取到的数据都是字符串，课程数为 '2' 而不是 2

csv 模块

csv 模块与字符串方法的对比

对比项	字符串方法（`join()` / `split()`）	`csv` 模块
数据内部含有逗号	数据被错误拆分	自动加双引号，读取时正确还原
行末换行符	写入时手动添加，读取时手动去除	自动处理
写入非字符串数据	引发 `TypeError`，须先转换	自动转换为字符串

练习题

- 第 1 题：写出下列程序的运行结果。若程序运行出错，写出出错前的输出，并指出错误类型

```
with open("courses.csv", "w", encoding="utf-8") as file:
    file.write("Python程序设计, 大学英语,高等数学\n")
with open("courses.csv", "r", encoding="utf-8") as file:
    content = file.read()
parts = content.split(",")
print(len(parts))
print(parts[1] == "大学英语")
print(len(parts[2]))
```

运行结果：

```
3
False
5
```

- "Python程序设计, 大学英语,高等数学\n"，其中共有 2 个逗号，按逗号拆分得到 3 个元素
- 第一个逗号后有一个空格，split(",") 不会去除它，第 2 个元素为 " 大学英语"，与 "大学英语" 不相等，结果为 False
- 读取时没有调用 strip()，第 3 个元素为 "高等数学\n"

练习题

- 第 2 题：写出下列程序的运行结果。若程序运行出错，写出出错前的输出，并指出错误类型

```
rows = [["张三", "2024001", "软件工程"], ["李四", "2024002", ""], ["王五", "2024003"]]
with open("students.csv", "w", encoding="utf-8") as file:
    for row in rows:
        file.write(",".join(row) + "\n")
with open("students.csv", "r", encoding="utf-8") as file:
    for line in file:
        fields = line.strip().split(",")
        print(fields[0], len(fields))
        print(fields[2] == "")
```

运行结果：

张三 3
False
李四 3
True
王五 2
IndexError: list index out of range

- 李四的专业为空字符串，join() 在其前面保留了一个逗号，拆分后仍为 3 列，第 3 列为空字符串，输出 True
- 王五的列表只有 2 个元素，写入后没有第 3 列，拆分得到 2 列，访问 fields[2] 时下标越界，引发 IndexError

练习题

- 第 3 题：写出下列程序的运行结果。若程序运行出错，写出出错前的输出，并指出错误类型

```
counts = [["课程", "人数"], ["大学英语", 2], ["高等数学", "1"]]
try:
    with open("count.csv", "w", encoding="utf-8") as file:
        for row in counts:
            file.write(",".join(row) + "\n")
except TypeError:
    print("写入出错")
with open("count.csv", "r", encoding="utf-8") as file:
    print(file.readlines())
```

运行结果：
写入出错
['课程,人数\n']

- 第二行 ["大学英语", 2] 中含有整数 2，join() 引发 TypeError
- with 语句在出错时已经关闭了文件，文件中只保留了出错前写入的第一行。readlines() 返回的列表中，元素包含末尾的 \n